Universidad Rey Juan Carlos

Trabajo de Fin de Grado

Enterprise RAG System:

Arquitectura Inteligente de Generación Aumentada por Recuperación con Modelos Locales

Autor:
Ángel Ballesteros

Tutor:
Nombre del Tutor

Grado en Ingeniería de Computadores
Curso 2025-2026
24 de febrero de 2026

Dedicatoria

A mi familia,
por su apoyo incondicional.

[Personaliza esta dedicatoria con tus propias palabras]

Agradecimientos

Este proyecto no habría sido posible sin el apoyo y la colaboración de numerosas personas e instituciones que han contribuido, directa o indirectamente, a su realización.

En primer lugar, quiero agradecer a mi tutor de TFG por su orientación académica y su disponibilidad a lo largo de todo el proceso de desarrollo. Sus revisiones y sugerencias han sido determinantes para dar forma al resultado final.

Agradezco también a la empresa Europavia por facilitar el entorno real de despliegue y las fuentes de datos corporativas que han servido como banco de pruebas del sistema. La oportunidad de desarrollar un proyecto con impacto práctico inmediato en un contexto profesional ha enriquecido enormemente la experiencia.

A mis compañeros de carrera, por las incontables horas compartidas de estudio, debugging y cafés. El camino se hace más llevadero cuando se recorre en buena compañía.

A mi familia, por su paciencia infinita durante los meses más intensos de desarrollo y redacción. Su apoyo incondicional ha sido el pilar fundamental sobre el que se sustenta todo lo demás.

Por último, quiero reconocer a la comunidad de código abierto, cuyos proyectos — Ollama, Qdrant, FastAPI, OpenWebUI, Playwright, PaddleOCR, LiteLLM, entre muchos otros — han hecho posible construir un sistema de esta envergadura sin dependencias comerciales. Este TFG es, en gran medida, un tributo a la filosofía del software libre.

Ángel Ballesteros
Madrid, 2026

Resumen

Este Trabajo de Fin de Grado presenta el diseño, desarrollo y despliegue de un sistema de Generación Aumentada por Recuperación (*Retrieval-Augmented Generation*, RAG) de ámbito corporativo, ejecutado íntegramente en infraestructura local (*on-premise*) y construido exclusivamente con tecnologías de código abierto.

El sistema permite a los empleados de una organización interactuar con sus documentos internos, páginas web y fuentes legislativas oficiales a través de una interfaz conversacional basada en lenguaje natural. La arquitectura emplea una pila de microservicios contenerizados con Docker que integra múltiples Modelos de Lenguaje Grande (LLMs) ejecutados localmente mediante Ollama, una base de datos vectorial Qdrant para la búsqueda semántica, y un agente enrutador inteligente denominado JARVIS que detecta automáticamente la intención del usuario y delega cada consulta al modo de operación más adecuado.

Entre las fuentes de información integradas se incluyen: Microsoft SharePoint (sincronización automática vía Microsoft Graph API), páginas web (scraping con Playwright y extracción NLP con Trafilatura), documentos PDF escaneados (OCR con PaddleOCR acelerado por GPU) y el Boletín Oficial del Estado (BOE) mediante el protocolo Model Context Protocol (MCP).

El sistema ha sido desplegado y validado en un entorno real con resultados satisfactorios, alcanzando tiempos de respuesta inferiores a 3.5 segundos y eliminando completamente las alucinaciones en modo RAG mediante técnicas de temperatura determinista y prompts restrictivos.

Palabras clave: RAG, LLM, inteligencia artificial, embeddings, base de datos vectorial, microservicios, Docker, Ollama, Qdrant, on-premise, código abierto.

Abstract

This Bachelor's Thesis presents the design, development and deployment of a corporate Retrieval-Augmented Generation (RAG) system, executed entirely on local infrastructure (on-premise) and built exclusively with open-source technologies.

The system enables employees of an organization to interact with their internal documents, web pages and official legislative sources through a conversational interface based on natural language. The architecture employs a stack of containerized microservices orchestrated with Docker, integrating multiple Large Language Models (LLMs) running locally via Ollama, a Qdrant vector database for semantic search, and an intelligent routing agent called JARVIS that automatically detects user intent and delegates each query to the most appropriate operation mode.

Integrated information sources include: Microsoft SharePoint (automatic synchronization via Microsoft Graph API), web pages (scraping with Playwright and NLP extraction with Trafilatura), scanned PDF documents (OCR with GPU-accelerated PaddleOCR) and the Spanish Official State Gazette (BOE) through the Model Context Protocol (MCP).

The system has been deployed and validated in a real production environment with satisfactory results, achieving response times under 3.5 seconds and completely eliminating hallucinations in RAG mode through deterministic temperature techniques and restrictive system prompts.

Keywords: RAG, LLM, artificial intelligence, embeddings, vector database, microservices, Docker, Ollama, Qdrant, on-premise, open source.

Índice general

Índice de figuras

Índice de tablas

Capítulo 1

Introducción

1.1. Motivación

En el contexto actual de las tecnologías de la información, el volumen de datos corporativos generados por las organizaciones crece exponencialmente. Plataformas como Microsoft SharePoint, sistemas de boletines oficiales (como el Boletín Oficial del Estado o BOE) y multitud de páginas web internas o externas, almacenan un conocimiento crítico y altamente especializado. No obstante, la búsqueda, recuperación y síntesis de esta información mediante métodos de búsqueda tradicional por palabras clave resulta, a menudo, ineficiente para los desarrolladores, arquitectos y empleados.

La llegada de los Modelos de Lenguaje Grande o Grandes Modelos de Lenguaje (LLMs) ha revolucionado la interacción humano-máquina mediante el uso de lenguaje natural. A pesar de sus capacidades narrativas y de razonamiento, estos modelos adolecen de un problema crítico: las *alucinaciones* o invenciones de información, al no poseer conocimiento actualizado o específico del dominio interno de una empresa. Es en este punto de intersección donde la Generación Aumentada por Recuperación (RAG, *Retrieval-Augmented Generation*) cobra vital importancia **lewis2020rag**.

Este Trabajo de Fin de Grado (TFG) tiene como motivación principal el diseño, desarrollo e integración de un sistema RAG corporativo (*Enterprise RAG System*), capaz de ingerir documentos desde múltiples fuentes heterogéneas (SharePoint, BOE, Webs) y proveer a los usuarios de una interfaz conversacional inteligente, privada y libre de alucinaciones.

1.1.1. Problemática del Shadow AI en Entornos Corporativos

La proliferación de herramientas de IA comerciales (ChatGPT, Google Gemini, Microsoft Copilot) en el ámbito laboral ha generado el fenómeno conocido como *Shadow AI*: el uso no autorizado de servicios de inteligencia artificial externos por parte de los empleados, frecuentemente sin el conocimiento del departamento de TI. Esta práctica introduce riesgos significativos:

- **Fuga de datos confidenciales:** Al copiar y pegar documentos internos en interfaces de terceros, la información sensible abandona el perímetro de seguridad de la organización.
- **Incumplimiento normativo:** En sectores regulados (defensa, sanidad, Administración Pública), el envío de datos a proveedores cloud estadounidenses puede violar el Reglamento General de Protección de Datos (RGPD) de la Unión Europea.
- **Inconsistencia de las respuestas:** Los modelos comerciales carecen de acceso al conocimiento especializado de la organización, produciendo respuestas genéricas o directamente incorrectas para consultas específicas del dominio.

El presente proyecto aborda directamente esta problemática mediante un sistema desplegado íntegramente en la infraestructura propia de la organización (*on-premise*), donde los datos nunca abandonan el servidor corporativo.

1.2. Objetivos del Proyecto

El objetivo principal de este trabajo es desarrollar un asistente conversacional avanzado sustentado en una arquitectura RAG 100 % *On-Premise* (de despliegue local), maximizando la confidencialidad de los datos internos y reduciendo la dependencia de APIs externas.

Para alcanzar este fin, se han definido los siguientes objetivos específicos:

1. **Diseño de Arquitectura Desacoplada:** Desarrollar un backend en Python (FastAPI) orquestado junto a infraestructuras de bases de datos vectoriales (Qdrant), bases de datos relacionales (PostgreSQL), motores locales de IA (Ollama) y balanceadores modulares de LLM (LiteLLM).

2. **Integración Multicanal:** Proveer al sistema de la capacidad de ingesta y sincronización automatizada conectando con la API Graph de Microsoft (SharePoint), extracción y comprensión mediante reconocimiento óptico de caracteres (OCR) ejecutado en GPU (PaddleOCR), y extracción web semántica estática y dinámica con *Playwright*.

3. **Resolución Normativa:** Integrar de forma nativa el BOE mediante el protocolo *Model Context Protocol* (MCP) utilizando un protocolo servidor-cliente estándar.

4. **Enrutamiento Inteligente:** Implementar un agente Python (*JARVIS Pipeline*) capaz de detectar la intención del usuario y derivar cada consulta al modo de operación más adecuado (RAG, búsqueda web, scraping, legal, visión o conversación general).

5. **Despliegue y Monitorización:** Empaquetar el sistema íntegramente mediante contenedores Docker con capacidades de control y recolección de métricas de uso y rendimiento del sistema mediante Prometheus y Grafana.

1.3. Alcance y Limitaciones

1.3.1. Alcance

El proyecto abarca el ciclo completo de desarrollo de un sistema RAG empresarial, desde la arquitectura hasta el despliegue en producción:

- Diseño e implementación de una arquitectura de microservicios contenerizada.
- Pipeline completo de ingesta documental con soporte para PDFs, HTML, imágenes y documentos de SharePoint.
- Motor de búsqueda semántica con embeddings y almacenamiento vectorial.
- Interfaz conversacional web con autenticación corporativa SSO.
- Integración con fuentes externas (SharePoint, BOE, internet).
- Stack de monitorización y observabilidad completo.

1.3.2. Limitaciones

Se identifican las siguientes limitaciones reconocidas:

- El sistema requiere una GPU NVIDIA con al menos 16 GB de VRAM para operar con rendimiento aceptable, lo que limita su ejecución en hardware doméstico.
- La integración con el BOE se encuentra en fase de prototipado técnico y depende enteramente de la disponibilidad de la API pública del BOE.
- El ajuste fino LoRA se realiza con datos sintéticos generados por el propio modelo base, lo que introduce un sesgo de distribución inherente.
- La evaluación de calidad se realiza de forma cualitativa; no se emplean benchmarks académicos formales como RAGAS o ARES por quedar fuera del alcance temporal del proyecto.

1.4. Contribuciones Principales

Las contribuciones técnicas concretas de este trabajo son:

1. Una **arquitectura de microservicios reproducible** definida declarativamente en Docker Compose con 12 servicios interconectados.
2. Un **agente enrutador inteligente** (JARVIS) que detecta 9 modos de operación distintos a partir de heurísticas semánticas.
3. Un **pipeline de ingesta multicanal** que unifica SharePoint, web scraping, OCR y carga manual en una única base de conocimiento vectorial.
4. Un **servidor MCP nativo** para la consulta del BOE Español con resolución semántica de nombres de leyes.
5. Un **framework de fine-tuning** con LoRA para adaptar modelos base al dominio empresarial específico.
6. Todo ello empaquetado como **software 100 % de código abierto** publicado en GitHub.

1.5. Estructura de la Memoria

El resto de este documento se estructura de la siguiente manera:

En el **Capítulo ??** se expone el estado actual del arte de los LLMs, los sistemas RAG, los modelos de embeddings, las bases de datos vectoriales, las técnicas de cuantización y el protocolo MCP.

El **Capítulo ??** describe la metodología ágil seguida durante el desarrollo, las iteraciones del proyecto, el entorno hardware, el stack tecnológico completo y las herramientas de gestión y documentación utilizadas.

El **Capítulo ??** profundiza en el diseño de la arquitectura general del sistema, detallando sus cinco capas (presentación, seguridad, orquestación, aplicación y datos), la orquestación Docker, los modelos de IA desplegados, el flujo de datos end-to-end y el stack de monitorización.

El **Capítulo ??** explica el proceso de implementación de cada subsistema principal (agente JARVIS, embeddings, web scraping, OCR, SharePoint, BOE y fine-tuning), acompañado de fragmentos de código reales, tablas de métricas de rendimiento y una evaluación cualitativa de los resultados.

Por último, el **Capítulo ??** recoge las conclusiones alcanzadas, las lecciones aprendidas durante el desarrollo e introduce las futuras líneas de investigación y expansión del trabajo.

Capítulo 2

Estado del Arte

En este capítulo se expone el marco teórico y tecnológico sobre el que se fundamenta el presente proyecto. Se analizarán las tecnologías punteras en el campo de los Modelos de Lenguaje Grande, las técnicas de recuperación de información que sustentan los sistemas RAG, y el ecosistema de herramientas de código abierto que hacen viable su despliegue sin dependencias externas.

2.1. Grandes Modelos de Lenguaje (LLMs)

Los Modelos de Lenguaje Grande (LLMs, por sus siglas en inglés) representan el estado del arte en el campo del Procesamiento del Lenguaje Natural (PLN). Impulsados por la arquitectura *Transformer* introducida por Vaswani et al. en 2017 **vaswani2017attention**, estos modelos utilizan mecanismos de atención para procesar secuencias verbales capturando el contexto y las relaciones semánticas complejas con un rendimiento sin precedentes.

2.1.1. Modelos Comerciales vs. Código Abierto

Mientras que sistemas comerciales como GPT-4 *(OpenAI)* o Claude 3 *(Anthropic)* lideran las métricas públicas, las opciones *Open Source* (Código Abierto) han experimentado un crecimiento abrumador. En este segmento, modelos de la familia LLaMA introducidos por Meta **touvron2023llama** han descentralizado el acceso a capacidades generativas avanzadas de varios miles de millones de parámetros. Su ventaja en despliegues corporativos resulta evidente: al poder ejecutarse de manera distribuida y en local o *on-premise*, el operador retiene la soberanía íntegra sobre los datos subyacentes e inputs operacionales ("prompts") insertados en todo el ciclo de inferencia.

Tabla 2.1: Comparativa de familias de LLMs relevantes para el proyecto.

Modelo	**Parámetros**	**Licencia**	**Local**	**Multilingüe**
GPT-4 (OpenAI)	∼1.7T	Propietaria	No	Sí
Claude 3 (Anthropic)	—	Propietaria	No	Sí
LLaMA 3.1 (Meta)	8–405B	Open Source	Sí	Sí
Qwen 2.5 (Alibaba)	0.5–72B	Open Source	Sí	Sí
Mistral (Mistral AI)	7–8x22B	Open Source	Sí	Sí
Gemma 2 (Google)	2–27B	Open Source	Sí	Parcial

En el presente proyecto se seleccionan los modelos LLaMA 3.1 8B de Meta y Qwen 2.5 de Alibaba. LLaMA 3.1 destaca por su rendimiento consolidado en tareas de instrucción y su amplio soporte comunitario. Qwen 2.5, por su parte, ofrece capacidades multilingües superiores y una arquitectura que se presta especialmente bien al ajuste fino con LoRA, como se discutirá en la Sección **??**.

2.2. Generación Aumentada por Recuperación (RAG)

Aunque los LLMs presentan altas métricas de razonamiento, sufren el fenómeno habitualmente conocido como "alucinación". Éste surge pues sus pesos estáticos están formados por distribuciones probabilísticas que no tienen la habilidad nativa de consultar bases de datos en tiempo real al generar cada *token*.

La técnica de *Retrieval-Augmented Generation* (RAG), formalizada académicamente por Lewis et al. en 2020 **lewis2020rag**, propone un enfoque híbrido. La pipeline RAG tradicional consta de dos fases:

1. **Recuperación (Retrieval):** Empleando modelos de *Embeddings*, se transforma la base de conocimiento y la consulta del usuario en vectores numéricos n-dimensionales que recogen representaciones espaciales semánticas. Tras esto, se buscan los vectores más similares matemáticamente en una Base de Datos Vectorial especializada.

2. **Generación (Generation):** El contexto recuperado proveniente de los vectores afines se le entrega estructuralmente formateado al modelo generativo bajo una instrucción "de sistema", obligando y restringiendo su actuación para fundamentar la respuesta empíricamente en el contexto hallado.

2.2.1. Variantes Avanzadas de RAG

La investigación reciente ha producido numerosas extensiones del paradigma RAG original:

- **Búsqueda Híbrida:** Combina la recuperación densa (basada en embeddings) con la recuperación dispersa (BM25 o TF-IDF), logrando capturar tanto la semántica profunda como las coincidencias léxicas exactas **chen2024bge**. El presente proyecto implementa esta variante.
- **Reranking:** Tras la fase de recuperación, un modelo especializado reordena los fragmentos candidatos según su relevancia contextual, mejorando la precisión del contexto inyectado.
- **RAG con Memoria:** Extiende el paradigma incorporando el historial de conversación como contexto adicional, permitiendo consultas de seguimiento coherentes.

2.3. Modelos de Embeddings

Los modelos de embeddings transforman cadenas de texto en vectores numéricos de dimensión fija, donde la proximidad geométrica entre vectores refleja la similitud semántica entre los textos originales. La métrica de similitud más empleada es la similitud del coseno:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{||\mathbf{a}|| \cdot ||\mathbf{b}||} \tag{2.1}$$

Entre las familias de modelos de embeddings más relevantes destacan:

- **SentenceTransformers reimers2019sentence**: Suite de modelos basados en la arquitectura BERT que producen embeddings de alta calidad optimizados para la similitud semántica. Su variante `paraphrase-multilingual-MiniLM-L12-v2`, utilizada en este proyecto, genera vectores de 384 dimensiones con soporte nativo para más de 50 idiomas.
- **OpenAI Embeddings:** Modelos comerciales como `text-embedding-3-small` ofrecen calidad superior pero requieren conexión a internet, lo que los descarta para despliegues puramente *on-premise*.
- **E5 y BGE:** Familias emergentes de embeddings de código abierto con rendimiento comparable a las alternativas comerciales, aunque con mayores requisitos de VRAM.

2.4. Bases de Datos Vectoriales

La eficiencia de un sistema RAG depende fundamentalmente de la capacidad de su base de datos vectorial para realizar búsquedas de vecinos más próximos (*k-NN*) en alta dimensionalidad con latencias mínimas.

Tabla 2.2: Comparativa de bases de datos vectoriales evaluadas para el proyecto.

Base de Datos	**Licencia**	**On-Premise**	**Filtrado**	**Híbrida**
Qdrant	Apache 2.0	Sí	Sí	Sí (BM25)
Pinecone	Propietaria	No (SaaS)	Sí	Parcial
Weaviate	BSD-3	Sí	Sí	Sí
ChromaDB	Apache 2.0	Sí	Limitado	No
Milvus	Apache 2.0	Sí	Sí	Sí

Se seleccionó **Qdrant** por su combinación de licencia abierta, soporte nativo de búsqueda híbrida (densa + dispersa), filtrado por metadatos y su rendimiento demostrado en producción con colecciones de millones de vectores. Su API REST nativa facilita la integración con el backend Python.

El índice HNSW (*Hierarchical Navigable Small World*) empleado por Qdrant ofrece búsquedas aproximadas de vecinos más próximos con complejidad logarítmica $O(\log n)$, haciéndolo escalable a grandes volúmenes documentales sin degradación significativa de la latencia de búsqueda.

2.5. Cuantización de Modelos

La ejecución local de LLMs en hardware generalista requiere técnicas de compresión que reduzcan el consumo de memoria VRAM. La cuantización consiste en representar los pesos del modelo con menor precisión numérica, pasando de coma flotante de 16 bits (FP16) a representaciones enteras de 8 o 4 bits.

- **Q8_0 (8 bits):** Reduce el consumo de VRAM a la mitad respecto a FP16 con una degradación de calidad inferior al 1 % en benchmarks estándar. Es la cuantización elegida para el modelo principal de conversación (LLaMA 3.1 8B).

- **Q4_0 (4 bits):** Reduce el consumo a una cuarta parte del modelo original. Se observan pérdidas medibles de calidad (2–5 %) que lo hacen adecuado para tareas de resumen donde la fidelidad absoluta es menos crítica.

- **GGUF:** Formato de archivo diseñado por el proyecto `llama.cpp` **gerganov2023ggml** que empaqueta los pesos cuantizados junto con los metadatos del modelo, permitiendo su carga eficiente en memoria.

Ollama, la plataforma de ejecución local empleada en este proyecto, utiliza internamente `llama.cpp` y el formato GGUF para servir los modelos cuantizados con aceleración GPU nativa.

2.6. Ajuste Fino Eficiente con LoRA

El ajuste fino (*fine-tuning*) permite adaptar un modelo pre-entrenado al dominio específico de una organización. Sin embargo, el ajuste fino completo de un modelo de 8 mil millones de parámetros requiere cantidades prohibitivas de VRAM y tiempo de computación.

La técnica LoRA (*Low-Rank Adaptation*), introducida por Hu et al. **hu2021lora**, propone una solución elegante: en lugar de modificar todos los pesos del modelo, se insertan pares de matrices de rango reducido A y B en las capas de atención del transformador, donde la actualización de pesos se aproxima como:

$$W' = W + \Delta W = W + BA \tag{2.2}$$

Con $B \in \mathbb{R}^{d \times r}$ y $A \in \mathbb{R}^{r \times k}$, donde $r \ll \text{mín}(d, k)$ es el rango de adaptación (típicamente $r = 8$ o $r = 16$). Esta factorización reduce drásticamente el número de parámetros entrenables (del orden del 0.1 % del total), permitiendo el ajuste fino en GPUs con 8–16 GB de VRAM en cuestión de horas.

2.7. Model Context Protocol (MCP)

El *Model Context Protocol* (MCP) **anthropic2024mcp** es un estándar abierto propuesto por Anthropic en noviembre de 2024 para formalizar la comunicación entre modelos de lenguaje y herramientas externas. MCP define una interfaz cliente-servidor donde:

- El **servidor MCP** publica un catálogo de herramientas (*tools*) con sus firmas de entrada/salida.
- El **cliente MCP** (habitualmente integrado en el agente) descubre las herramientas disponibles y las invoca proporcionando los argumentos requeridos.

Este protocolo estandariza lo que anteriormente requería implementaciones ad-hoc para cada integración, facilitando la interoperabilidad. En el presente proyecto, se implementa un servidor MCP nativo con la librería `fastmcp` de Python que expone las funciones de consulta a la API del BOE como herramientas MCP consumibles por JARVIS.

2.8. Tecnologías de Soporte del Ecosistema

Recientemente, diversos ecosistemas facilitan en la actualidad el puente entre motores de ejecución *Backend* y la lógica RAG:

- **Ollama:** Se ha consolidado como un proyecto clave para instanciar, cargar en VRAM y exponer inferencia local multiplataforma de manera compatible y eficiente con hardware generalista, a menudo empleando cuantización sub 8-bit.
- **LiteLLM:** Proxy unificado que traduce las APIs propietarias de más de 100 proveedores de LLM al formato estándar de OpenAI, permitiendo cambiar de modelo sin alterar el código cliente.
- **LangChain y LlamaIndex:** Bibliotecas estándar en Python para orquestar la conexión de fuentes de datos (desde archivos locales hasta almacenamiento en nube Azure/AWS), gestionar troceado semántico o *chunking* de texto e inyectarlo ordenadamente al motor LLM vía su API correspondiente.
- **FastAPI:** Framework web moderno de Python basado en tipado estricto y validación automática con Pydantic, que genera documentación OpenAPI/Swagger de forma nativa. Seleccionado por su rendimiento asíncrono y su ecosistema de inyección de dependencias.

Sin embargo, las alternativas comerciales genéricas continúan pecando de ser soluciones "caja blanca" o, contrariamente, de un elevado coste por *tokens* ingeridos cuando se trata de conectar sistemas altamente regulados o heterogéneos (sistemas integrados con Webhooks directos a instancias locales). Esto resalta la necesidad de desarrollar plataformas desacopladas de código abierto que permitan su extensión en el tejido empresarial, tal y como se asienta el presente fundamento del proyecto.

Capítulo 3

Metodología

En este capítulo se describe el proceso de desarrollo seguido, las herramientas empleadas, el entorno hardware en el que se despliega el sistema y la estrategia de documentación adoptada.

3.1. Proceso de Desarrollo

El diseño e implementación de este Trabajo de Fin de Grado se ha conducido siguiendo un marco de trabajo de desarrollo ágil (*Agile*), caracterizado por entregas iterativas e incrementales. Dada la naturaleza cambiante y el rápido progreso tecnológico inherente a al campo de la Inteligencia Artificial y los Modelos de Lenguaje Grande (LLMs) durante el transcurso del tiempo del desarrollo, la flexibilidad para pivotar decisiones de enrutamiento o cambiar contenedores y versiones del modelo resultó crítica.

Se estableció un ciclo de vida con las siguientes fases:

1. **Elicitación y Análisis de Requisitos:** Identificación de fuentes de información clave (SharePoint, páginas web corporativas y el BOE), así como las restricciones de privacidad requeridas por un despliegue puramente *on-premise*.

2. **Diseño de Arquitectura:** Planificación de la infraestructura basada en microservicios contenerizados y selección del stack tecnológico idóneo.

3. **Implementación Incremental:** Programación progresiva de los módulos de ingesta, OCR, embeddings, búsqueda de Qdrant, backend y, por último, de la interfaz de usuario de OpenWebUI.

4. **Pruebas y Validación:** Validación funcional del enrutamiento inteligente y de las capacidades del RAG mediante métricas comparativas empíricas y

experimentación cualitativa.

3.1.1. Iteraciones de Desarrollo

El proyecto se dividió en iteraciones incrementales de tipo sprint, cada una enfocada en un módulo funcional del sistema. La Tabla **??** resume las iteraciones de desarrollo junto con sus entregables principales y las horas dedicadas estimadas.

Tabla 3.1: Iteraciones principales del desarrollo, entregables y horas dedicadas.

Sprint	**Período**	**Entregable Principal**	**Horas**
1	Oct 2025	Infraestructura Docker, PostgreSQL, Qdrant, Ollama. Pruebas de concepto con LLaMA.	25 h
2	Nov 2025	Backend FastAPI con pipeline RAG básico. Ingesta de PDFs y generación de embeddings.	35 h
3	Dic 2025	Integración SharePoint (MSAL/Graph API). Motor OCR PaddleOCR con GPU.	30 h
4	Ene 2026	Agente JARVIS Pipeline. Web scraping con Playwright. Búsqueda DuckDuckGo.	40 h
5	Feb 2026	Servidor MCP del BOE. Fine-tuning LoRA (Qwen). Stack Prometheus/Grafana.	30 h
		Total desarrollo técnico	**160 h**

3.1.2. Diagrama de GANTT

La distribución temporal del proyecto se representa en el diagrama de GANTT de la Figura **??**. Cada barra horizontal corresponde a una fase o sprint de desarrollo, con las dependencias marcadas implícitamente por la secuencia temporal.

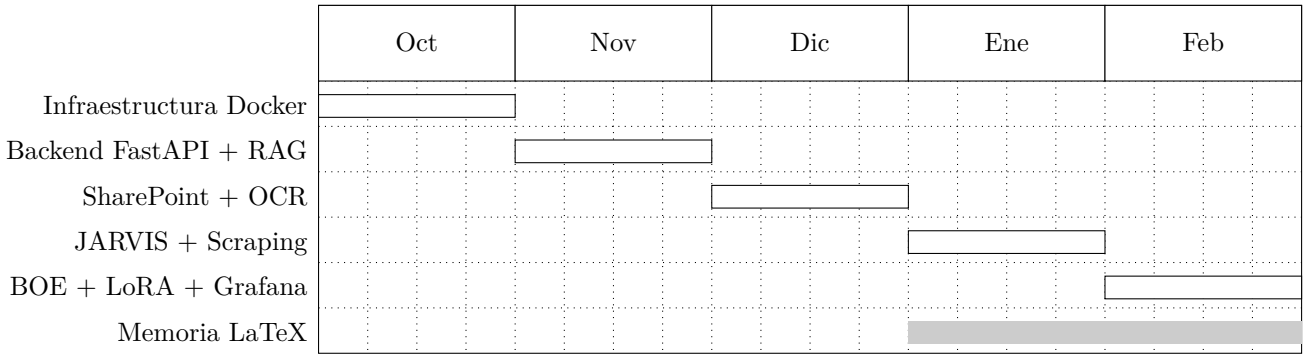

Figura 3.1: Diagrama de GANTT del proyecto.

3.2. Entorno Hardware

El sistema fue diseñado para ejecutarse en un servidor físico on-premise con las siguientes especificaciones:

Tabla 3.2: Especificaciones del entorno hardware de despliegue.

Componente	**Especificación**
Sistema Operativo	Ubuntu Server 22.04 LTS
CPU	AMD / Intel x86_64 (multi-core)
RAM	32 GB DDR4 mínimo
GPU	NVIDIA con $\geq$ 16 GB VRAM (p.ej. RTX 4090, A4000)
CUDA Toolkit	12.x
Almacenamiento	SSD NVMe $\geq$ 500 GB
Docker Engine	24.x con NVIDIA Container Toolkit

La GPU es el recurso más crítico del sistema, dado que los tres modelos de lenguaje, el motor OCR y los modelos de visión comparten la misma tarjeta gráfica. La gestión eficiente de la VRAM mediante cuantización (Q8, Q4) y la carga/descarga dinámica de modelos en Ollama resultó una decisión arquitectónica determinante.

3.3. Stack Tecnológico

La selección de tecnologías priorizó tres criterios: licencia de código abierto, ejecutabilidad local sin dependencias de nube, y madurez del ecosistema. La Tabla **??** enumera la totalidad de tecnologías empleadas.

Tabla 3.3: Stack tecnológico completo del proyecto.

Categoría	**Tecnología**	**Versión / Detalle**
Lenguajes	Python	3.11
	YAML / Bash	Configuración y scripts
Framework Web	FastAPI	0.104+
Motor de IA Local	Ollama	latest
Proxy de Modelos	LiteLLM	latest
Frontend Chat	OpenWebUI	main
BD Vectorial	Qdrant	latest
BD Relacional	PostgreSQL	16 con pgvector
Caché	Redis	7 Alpine
Objetos	MinIO	latest
Web Scraping	Playwright	Chromium
Extracción NLP	Trafilatura	latest
OCR	PaddleOCR	2.x con CUDA
Embeddings	SentenceTransformers	MiniLM-L12-v2
Autenticación	Azure AD (OIDC)	MSAL Python
SharePoint	Microsoft Graph API	v1.0
Métricas	Prometheus	2.x
Dashboards	Grafana	latest
GPU Exporter	NVIDIA DCGM	latest
Contenerización	Docker + Compose	24.x
Control de Versiones	Git + GitHub	—
Proxy Inverso	NGINX	Alpine

3.4. Herramientas y Control de Versiones

Para la correcta orquestación colaborativa del proyecto se utilizó el sistema de control de versiones Git, albergando el repositorio de código principal en GitHub[1]. El seguimiento de incidencias, errores (*bugs*) y nuevas características (*features*) se coordinó siguiendo la metodología y ramificación *Git Flow*, segregando los entornos de producción (*main*) del trabajo en progreso con ramas independientes.

El entorno de desarrollo integrado (IDE) empleado primordialmente fue Visual

[1] https://github.com/acidxlemons/TFG-JARVIS

Studio Code en Windows, acoplado al subsistema WSL o contenedores Docker en remoto para igualar la paridad funcional con el entorno de despliegue real Linux, evitando fricciones a nivel de sistema de archivos o librerías específicas compiladas para CUDA/NVIDIA.

3.4.1. Integración Continua con GitHub Actions

Se configuró un flujo de integración continua (CI) mediante GitHub Actions para automatizar el despliegue de la documentación del proyecto en GitHub Pages. Cada vez que se realiza un *push* a la rama `main`, el workflow compila y publica automáticamente la landing page del proyecto, accesible públicamente en https://acidxlemons.github.io/TFG-JARVIS/.

3.5. Estimación de Presupuesto

Aunque el proyecto se ha desarrollado íntegramente con software de código abierto, se estiman los costes asociados al despliegue profesional del sistema si se deseara replicar o escalar:

Tabla 3.4: Estimación de presupuesto para el despliegue del sistema.

Concepto	**Coste Unitario**	**Coste Total**
Servidor GPU (NVIDIA RTX 4090 24GB)	—	2.500 €
Servidor torre (CPU, RAM 64GB, SSD)	—	1.500 €
Licencias de software	—	0 € (Open Source)
Dominio y certificados SSL	15 €/año	15 €
Electricidad servidor 24/7 (año)	∼50 €/mes	600 €
Horas de desarrollo (200h × 25 €/h)	25 €/h	5.000 €
	Total estimado	**9.615 €**

Cabe destacar que el coste de licencias de software es **cero euros**, dado que todas las tecnologías empleadas son de código abierto. Esto representa una ventaja competitiva significativa frente a soluciones comerciales como Microsoft Copilot o AWS Bedrock, cuyos costes recurrentes por token procesado pueden escalar rápidamente en entornos de uso intensivo.

3.6. Gestión de Riesgos

Dado el carácter experimental del proyecto y la velocidad de evolución del campo de los LLMs, se identificaron los siguientes riesgos técnicos y sus respectivas mitigaciones:

Tabla 3.5: Riesgos técnicos identificados y estrategias de mitigación.

Riesgo	**Mitigación**
Agotamiento de VRAM con múltiples modelos cargados	Cuantización Q4/Q8 y carga dinámica bajo demanda en Ollama
Obsolescencia de un modelo de IA	Arquitectura desacoplada vía LiteLLM que permite cambiar modelos sin alterar código
Fallo de la GPU o driver CUDA	Fallback automático a CPU en PaddleOCR y SentenceTransformer
Cambios en la API de SharePoint	Capa de abstracción con MSAL que aísla los cambios de endpoints
Pérdida de datos en Qdrant	Backup periódico de colecciones y réplica en MinIO S3

3.7. Impacto de Asignaturas Cursadas en la Carrera

El desarrollo de este TFG ha sido posible gracias a la base de conocimientos adquirida en múltiples asignaturas del Grado en Ingeniería Informática, aunque también ha revelado áreas donde fue necesario un aprendizaje autónomo complementario.

3.7.1. Conocimientos Aplicados del Grado

- **Ingeniería del Software:** Los patrones de diseño, la separación de responsabilidades y la metodología ágil aprendidos en esta asignatura fueron directamente aplicables al diseño de la arquitectura de microservicios y a la gestión iterativa del proyecto.

- **Bases de Datos:** Los fundamentos de SQL, diseño relacional y normalización permitieron la configuración y administración de PostgreSQL, así como la

comprensión del modelo de datos empleado por OpenWebUI para almacenar conversaciones y usuarios.

- **Redes de Computadores:** El conocimiento de protocolos TCP/IP, DNS, proxies inversos y certificados TLS fue indispensable para la configuración de NGINX, la red Docker interna y la securización del tráfico HTTPS.

- **Sistemas Operativos:** La gestión de contenedores Docker, volúmenes, procesos, y la interacción con drivers NVIDIA/CUDA se nutre directamente de los conceptos de virtualización, gestión de memoria y planificación de procesos.

- **Inteligencia Artificial:** Los fundamentos de aprendizaje automático, redes neuronales y procesamiento del lenguaje natural proporcionaron la base teórica para comprender el funcionamiento de los LLMs y los modelos de embeddings.

- **Programación Web:** El desarrollo de APIs REST con FastAPI, la gestión de peticiones HTTP asíncronas y el manejo de formatos JSON se basan en los conocimientos de programación web adquiridos durante la carrera.

3.7.2. Conocimientos Adquiridos de Forma Autónoma

- **Modelos de Lenguaje Grande (LLMs):** La comprensión profunda de la arquitectura Transformer, los mecanismos de atención, la tokenización y las técnicas de cuantización no forman parte del Plan de Estudios actual y fueron adquiridos mediante documentación técnica y papers académicos.

- **Bases de Datos Vectoriales:** El concepto de embeddings, búsqueda por similitud del coseno e índices HNSW no se cubre en las asignaturas de Bases de Datos convencionales.

- **Fine-Tuning con LoRA:** Las técnicas de ajuste fino paramétrico eficiente requirieron inmersión en bibliografía especializada **hu2021lora**.

- **Docker avanzado y GPU passthrough:** La orquestación de contenedores con acceso a GPU mediante `nvidia-container-toolkit` excede significativamente los contenidos del grado.

- **Integración con Microsoft Graph y MSAL:** La autenticación OAuth 2.0 con flujo de credenciales de cliente y la interacción con la API Graph de Microsoft requirieron aprendizaje específico de la plataforma Azure.

3.8. Estrategia de Documentación

La documentación es una de las esquinas vertebradoras del proyecto, debido a la alta mantenibilidad exigida. *Markdown* sirvió como estándar *de facto* para manuales de despliegue, la guía técnica de arquitectura y los documentos de referencia del repositorio.

Se elaboraron los siguientes documentos técnicos complementarios a esta memoria:

- `README.md`: Visión general del proyecto con instrucciones de despliegue rápido.
- `TECHNICAL_ARCHITECTURE.md`: Documentación detallada de la arquitectura de microservicios.
- `USER_GUIDE.md`: Guía de usuario con ejemplos de uso de cada funcionalidad.
- `DEPLOYMENT_CHECKLIST.md`: Lista de verificación paso a paso para el despliegue en producción.

Adicionalmente, se configuró un ecosistema de documentación de APIs de manera autónoma cumpliendo los estándares OpenAPI en el motor web de FastAPI. Esto generó de forma automática esquemas visuales de Swagger (`/docs` y `/redoc`), vitales para la fase de prototipado y depuración del tráfico de microservicios e ingestores hacia el motor central.

3.8.1. Redacción de la Memoria

La redacción, edición y organización de la presente memoria del proyecto requirió aproximadamente **40 horas**, incluyendo la recopilación de referencias bibliográficas, la generación de tablas y diagramas, y la revisión del formato académico según las normativas de la Universidad Rey Juan Carlos.

Capítulo 4

Arquitectura del Sistema

El diseño lógico de este proyecto orquesta una extensa pila tecnológica (*Stack*) desacoplada para garantizar escalabilidad, monitorización exhaustiva y una separación de responsabilidades estricta. La arquitectura general emplea una metodología *microservices-oriented* (orientada a microservicios) empaquetada globalmente mediante contenedores Docker con un manifiesto *Docker Compose*.

4.1. Vista General en Capas

El sistema se diseña partiendo de una distinción clásica entre la fachada de interfaz visible, la lógica de control intermedia y múltiples bases de datos de apoyo especializadas. Se pueden identificar cinco capas claramente diferenciadas, desde la interfaz hasta la persistencia.

4.1.1. Capa de Presentación (Frontend)

El único punto de acceso de cara al usuario está gobernado por el módulo OpenWebUI e iterado a través de un proxy inverso Nginx, asegurando una entrada HTTPS fortificada con puertos redireccionados aislados. Esta capa contiene el componente web que implementa la pasarela para interaccionar nativamente con el motor, además de gestionar la Autenticación Única (SSO, *Single Sign-On*) del Azure Active Directory.

OpenWebUI fue seleccionado frente a alternativas como Gradio, Streamlit o ChatUI por su compatibilidad nativa con Ollama, su sistema de plugins (*Pipelines*) que permite inyectar lógica personalizada sin modificar el código fuente del frontend, y su gestión de usuarios y conversaciones integrada de serie con persistencia en PostgreSQL.

4.1.2. Capa de Seguridad Perimetral

El proxy inverso NGINX actúa como la primera barrera de seguridad del ecosistema. Escucha exclusivamente en el puerto **8443** con un certificado TLS/SSL autofirmado para el tráfico interno corporativo. Su configuración principal incluye:

- **Terminación TLS:** Todo el tráfico entre el navegador del usuario y el servidor se cifra. Los microservicios internos se comunican en texto plano dentro de la red Docker aislada.
- **Proxy pass transparente:** Las peticiones HTTPS se redireccionan internamente a OpenWebUI (puerto 8080) preservando las cabeceras `X-Forwarded-For` y `X-Real-IP` para la trazabilidad.
- **WebSocket upgrade:** Se habilita la mejora de protocolo a WebSocket, requisito indispensable para el streaming de tokens en tiempo real (*Server-Sent Events*).

4.1.3. Capa de Orquestación y Enrutamiento Inteligente

De manera subyacente a OpenWebUI, un agente enrutador denominado *JARVIS Pipeline* recibe cada evento. Elaborado sobre Python nativo, JARVIS no emite inferencias instantáneas, sino que evalúa la intención del usuario a nivel semántico (por ejemplo, detectar la mención de hipervínculos para un *scraping* asíncrono, deducir la motivación de emplear el corpus documental mediante la activación del *RAG* explícito, o dictaminar la necesidad de contactar al BOE). JARVIS orquesta flujos mediante peticiones asíncronas HTTP a la capa de Aplicación según los parámetros resueltos.

El pipeline opera como un *middleware* interpuesto entre la interfaz de usuario y los modelos de lenguaje. OpenWebUI envía a JARVIS cada mensaje del usuario como un evento HTTP con el texto, los adjuntos (si los hay) y los metadatos del usuario. JARVIS analiza el contenido, selecciona el modo de operación apropiado y delega la ejecución al microservicio correspondiente.

4.1.4. Capa de Aplicación (Backend)

El cerebro lógico principal del proyecto se sustenta en tres microservicios interconectados:

- **API Backend (FastAPI):** Servicio central que atiende todas las operaciones RAG. Implementa los endpoints REST para la ingesta de documentos

(`/ingest`), la búsqueda semántica (`/chat`), el web scraping (`/scrape`), la consulta de documentos indexados (`/documents`) y la exportación de métricas para Prometheus (`/metrics`). Internamente orquesta la cadena de procesamiento: chunking semántico, generación de embeddings con SentenceTransformer, almacenamiento en Qdrant y formateo de prompts contextuales para el LLM.

- **Indexer API:** Responsable explícito de sondear periódicamente los orígenes asíncronos. Realiza la conexión a *Microsoft Graph* monitorizando novedades en los ficheros de SharePoint o las peticiones en carpetas locales. Llama automáticamente al motor OCR (PaddleOCR) habilitado sobre GPU si detecta PDFs escaneados a gran escala o ficheros imposibles de parsear semánticamente por texto plano. Funciona de forma completamente autónoma mediante un cron interno configurable.

- **LiteLLM Proxy:** Intermediario en el control de acceso a los Modelos Locales, unificando endpoints de Ollama bajo una API compatible con el estándar de OpenAI. Registra tiempos de servicio, habilita cachés perimetrales sobre Redis para evitar múltiples llamadas de red de las respuestas computadas y aplica reglas de balanceo de carga cuando varios modelos están disponibles simultáneamente.

4.1.5. Capa de Datos

La persistencia de memoria se divide drásticamente en un entorno distribuido en cuatro nodos independientes en la pila:

1. **Qdrant — Base de Datos Vectorial:** Administra las colecciones `documents` y `webs` almacenando incrustaciones generadas (*Embeddings* a 384 dimensiones) indexadas por índices hiper-dimensionales afines HNSW (*Hierarchical Navigable Small World*). Despliega además en conjunción estrategias de filtrado disperso como BM25 para búsqueda híbrida. La colección de documentos se segmenta por departamento, permitiendo el control de acceso granular.

2. **PostgreSQL con pgvector:** Garantiza consistencia relacional ACID en cuanto a persistir el árbol histórico de interacciones (conversaciones, turnos de palabras, usuarios registrados y trazabilidad del SSO). La extensión pgvector se mantiene instalada como capacidad futura para operaciones vectoriales secundarias.

3. **Redis — Almacén en Memoria:** Interviene como mecanismo de caché para aligerar peticiones al LLM de respuestas repetitivas o idénticas. Cada respuesta cacheada tiene un *Time To Live* (TTL) configurable de 1 hora, tras el cual se invalida automáticamente.

4. **MinIO S3 — Almacenamiento de Objetos:** Preserva una réplica aislada de todos los objetos en formato archivo binario en el sistema, dotando de resistencia a la configuración frente a fallos y habilitando un sistema de backup independiente de la base de datos.

4.2. Orquestación con Docker Compose

La totalidad del ecosistema se define declarativamente en un único fichero `docker-compose.yml`. Este manifiesto YAML describe los 12 servicios del sistema, sus dependencias, volúmenes persistentes, variables de entorno y políticas de reinicio.

Tabla 4.1: Servicios definidos en el manifiesto Docker Compose.

Servicio	**Imagen / Build**	**Puerto Expuesto**
nginx	nginx:alpine	8443 (HTTPS)
open-webui	open-webui:main	8080 (interno)
pipelines	Custom (Dockerfile)	9099 (interno)
rag-backend	Custom (Dockerfile)	8000 (interno)
indexer	Custom (Dockerfile)	8001 (interno)
ollama	ollama/ollama:latest	11434 (interno)
litellm	litellm/litellm:main	4000 (interno)
qdrant	qdrant/qdrant	6333 (interno)
postgres	postgres:16	5432 (interno)
redis	redis:7-alpine	6379 (interno)
mcp-boe-server	Custom (Dockerfile)	8010 (interno)
minio	minio/minio	9000 (interno)

Todos los servicios comparten una red Docker interna de tipo *bridge* denominada `tfg-network`. Únicamente el servicio de Nginx expone un puerto al exterior (8443), garantizando que ningún microservicio interno sea accesible directamente desde fuera del servidor.

Los servicios que requieren acceso a GPU (Ollama, PaddleOCR) utilizan la directiva `deploy.resources.reservations.devices` de Docker Compose para reservar dispositivos NVIDIA mediante el runtime `nvidia-container-toolkit`.

4.3. Modelos de Inteligencia Artificial Desplegados

El sistema opera simultáneamente con seis modelos de IA ejecutados íntegramente en hardware local. La selección de modelos se realizó en base a un equilibrio entre capacidad generativa, consumo de VRAM y velocidad de inferencia.

Tabla 4.2: Inventario completo de modelos IA desplegados y su configuración en LiteLLM.

Alias LiteLLM	**Modelo Real**	**Función**	**VRAM**
`llama3.1-8b`	llama3.1:8b-instruct-q8_0	Chat y RAG general	∼8 GB
`llama3.1-summarize`	llama3.1:8b-instruct-q4_0	Resúmenes ligeros	∼4 GB
`JARVIS`	rag-qwen-ft:latest	RAG corporativo (FT)	∼5 GB
—	qwen2.5vl:7b	Visión multimodal	∼5 GB
—	llava:13b	Visión (legacy)	∼8 GB
—	MiniLM-L12-v2	Embeddings (CPU)	0 GB

La cuantización Q8_0 del modelo LLaMA 3.1 8B ofrece un compromiso óptimo entre calidad y velocidad: reduce el consumo de VRAM un 50 % respecto a la versión FP16 con una degradación de calidad inferior al 1 % en benchmarks estándar. El modelo Q4_0 se reserva para tareas de resumen donde la fidelidad absoluta es menos crítica.

4.4. Flujo de Datos End-to-End

Para ilustrar la cohesión del sistema, se describe el flujo completo que atraviesa una consulta RAG típica desde que el usuario la formula hasta que recibe la respuesta:

1. El **usuario** escribe su pregunta en OpenWebUI y pulsa Enter.

2. **NGINX** recibe la petición HTTPS y la redirecciona internamente a OpenWebUI.

3. **OpenWebUI** envía el mensaje al pipeline JARVIS mediante una llamada HTTP POST al puerto 9099.

4. **JARVIS** analiza el texto, detecta la intención RAG y construye una petición al Backend con el query del usuario y las colecciones autorizadas para su departamento.

5. El **Backend** genera el embedding de la consulta con SentenceTransformer, ejecuta una búsqueda híbrida (HNSW + BM25) en Qdrant y recupera los k fragmentos más relevantes.

6. El Backend compone un *prompt* estructurado que incluye el contexto recuperado, la pregunta original y las instrucciones de sistema, y lo envía a LiteLLM.

7. **LiteLLM** verifica si existe una respuesta cacheada en Redis. Si no, delega al modelo apropiado en Ollama.

8. **Ollama** ejecuta la inferencia en GPU y devuelve los tokens generados en streaming.

9. La respuesta fluye de vuelta por la cadena: LiteLLM → Backend → JARVIS → OpenWebUI → NGINX → navegador del usuario.

Todo el flujo se completa típicamente en menos de 3.5 segundos para consultas RAG estándar, con el primer token visible para el usuario en menos de 1.2 segundos gracias al streaming.

4.5. Redes, Seguridad y Monitorización

4.5.1. Aislamiento de Red

Toda la malla de nodos convive dentro de un plano interno `tfg-network`, un modo de puente o *bridge network* gestionado por el motor host de Docker, impidiendo a entes externos o atacantes conectar con puertos de base de datos no expuestos frontalmente. Esto significa que servicios como PostgreSQL, Redis y Qdrant **no son accesibles** desde fuera del servidor bajo ninguna circunstancia.

4.5.2. Autenticación SSO con Azure AD

OpenWebUI se configura con el protocolo OpenID Connect (OIDC) apuntando al *tenant* corporativo de Azure Active Directory. Los usuarios se autentican con sus credenciales corporativas sin necesidad de crear una cuenta separada. Los tokens JWT emitidos por Azure AD se validan en cada petición y sus atributos (nombre, correo, grupos) se utilizan para el control de acceso departamental en JARVIS.

4.5.3. Stack de Observabilidad

Para consolidar la resiliencia en un marco empresarial de alto rendimiento, se integran los siguientes componentes de monitorización:

- **Prometheus:** Servidor de métricas temporales que scrapea activamente los endpoints `/metrics` del Backend (FastAPI) cada 15 segundos, almacenando series temporales de latencia, throughput, errores y uso de recursos.
- **NVIDIA DCGM Exporter:** Exportador especializado que captura la saturación paralela de las GPUs de NVIDIA: consumo de memoria GDDR6 (VRAM), utilización de los *CUDA cores*, temperatura y potencia. Estas métricas son scrapeadas por Prometheus y resultan críticas para detectar cuellos de botella en la inferencia.
- **Grafana:** Portal de visualización que consume las métricas de Prometheus y las presenta en dashboards interactivos. Se configuraron tres dashboards principales: uno para métricas del Backend (latencia, peticiones, errores), otro para métricas de GPU (VRAM, temperatura, carga) y un tercero para métricas de base de datos.
- **pgAdmin 4:** Interfaz web de administración de PostgreSQL que permite auditar directamente las tablas de conversaciones, usuarios y sesiones almacenadas por OpenWebUI.

Capítulo 5

Implementación y Resultados

Este capítulo constituye el núcleo técnico del proyecto. Se describirá en profundidad la implementación funcional de cada uno de los subsistemas del *Enterprise RAG System*, acompañando las explicaciones con fragmentos de código reales extraídos del repositorio, tablas de configuración y ejemplos prácticos de uso.

5.1. Arquitectura del Agente Enrutador JARVIS

El agente **JARVIS** es la pieza central de orquestación del sistema. Implementado como una clase Python `Pipeline` compatible con la interfaz de plugins de Open-WebUI, actúa como un enrutador inteligente que analiza cada mensaje del usuario y decide a qué modo de operación derivar la petición.

5.1.1. Detección de Intenciones

El mecanismo de detección de intenciones se implementa mediante un sistema de reglas heurísticas basado en coincidencia de palabras clave y expresiones regulares, evaluadas en un orden estrictamente prioritario. Se definen las siguientes diez categorías de intención:

Tabla 5.1: Modos de operación del agente JARVIS y sus criterios de activación.

Modo	**Acción**	**Ejemplo de activación**
Visión/OCR	`ocr`	Imagen adjunta en el chat
RAG Documental	`rag`	"¿Qué dice el manual de calidad?"
Web Search	`web_search`	"Busca en internet sobre Docker"
Web Scraping	`web_scrape`	Incluir una URL en el mensaje
BOE Legal	`boe_search`	"Busca en el BOE la ley de datos"
Listar Docs	`list_docs`	"/docs" o "¿Qué documentos tienes?"
Status	`check_status`	"¿Cómo va la indexación?"
Ayuda	`help`	"¿Qué puedes hacer?"
Conversación	`chat`	Cualquier otro mensaje

La evaluación sigue un protocolo de prioridades estricto: primero se verifican las condiciones más específicas (como la presencia de archivos adjuntos o imágenes), y se desciende progresivamente hacia las heurísticas más genéricas. Este diseño evita que un modo ambiguo como Web Search capture consultas que deberían derivarse al RAG.

5.1.2. Memoria de Sesión por Usuario

JARVIS mantiene un estado volátil en memoria para cada usuario identificado por su dirección de correo electrónico. Este estado incluye tres estructuras fundamentales:

- `_user_image_memory`: Almacena la última imagen enviada por el usuario en formato Base64, permitiendo preguntas de seguimiento como "¿Qué pone en la imagen anterior?" sin necesidad de reenviarla.

- `_user_web_memory`: Retiene el contenido extraído de la última URL analizada, habilitando consultas contextuales como "Resume la web que acabamos de mirar".

- `_user_pending_decision`: Gestiona decisiones pendientes cuando el sistema detecta que una URL ya fue indexada previamente y ofrece al usuario la opción de reutilizar el contenido existente o volver a scrapearla.

5.1.3. Control de Acceso Basado en Grupos

El pipeline integra un sistema de permisos multi-departamento que segmenta el acceso a las colecciones vectoriales según los grupos de Azure Active Directory del usuario. Al recibir una petición, JARVIS extrae los grupos del objeto `user_data` proporcionado por OpenWebUI y los mapea a colecciones de Qdrant mediante un diccionario configurable:

```
DEPARTMENT_MAPPING: Dict[str, str] = {
    "CIVEX2": "documents_CIVEX2",
    "7573b3c1-eeb0-4e3b-8c41-08b749a1dffd": "documents_CIVEX2",
}
DEFAULT_COLLECTION: str = "documents"
```

Listing 5.1: Configuración del mapeo departamental en JARVIS.

Si el usuario no pertenece a ningún grupo específico, accede únicamente a la colección por defecto. Los usuarios con rol de administrador acceden automáticamente a todas las colecciones. Además, se implementa una caché de permisos con un *Time-To-Live* (TTL) configurable de 300 segundos para minimizar las llamadas repetitivas a la API de Microsoft Graph.

5.2. Pipeline de Embeddings y Búsqueda Semántica

5.2.1. Modelo de Embeddings

El sistema utiliza el modelo `paraphrase-multilingual-MiniLM-L12-v2` de la librería *SentenceTransformers*, que genera vectores densos de 384 dimensiones optimizados para la similitud del coseno multilingüe. Este modelo fue seleccionado por los siguientes criterios técnicos:

Tabla 5.2: Comparativa de modelos de embeddings evaluados.

Modelo	**Dimensiones**	**Multilingüe**	**Tamaño (MB)**
all-MiniLM-L6-v2	384	No	80
paraphrase-multilingual-MiniLM-L12-v2	384	Sí	420
all-mpnet-base-v2	768	No	420
multilingual-e5-large	1024	Sí	2200

La ejecución se fuerza explícitamente en CPU dentro del contenedor del backend

para evitar conflictos de versiones CUDA con PaddleOCR, que monopoliza la GPU para tareas de reconocimiento óptico:

```
class EmbeddingGenerator:
    def __init__(self, model_name="paraphrase-multilingual-MiniLM-L12-v2"):
        self.model = SentenceTransformer(model_name, device="cpu")
        self.vector_dim = self.model.get_sentence_embedding_dimension()
```

Listing 5.2: Inicialización del generador de embeddings forzando ejecución en CPU.

5.2.2. Estrategia de Chunking

Los documentos ingeridos se segmentan en fragmentos (*chunks*) de tamaño configurable, típicamente entre 512 y 1024 tokens, con un solapamiento (*overlap*) del 10–15 % entre fragmentos consecutivos. Este solapamiento es vital para mantener la coherencia semántica en párrafos que cruzan los límites de ventana.

5.2.3. Búsqueda Híbrida en Qdrant

La recuperación de contexto combina dos estrategias complementarias:

1. **Búsqueda densa HNSW:** Emplea los vectores de 384 dimensiones para localizar los fragmentos semánticamente más próximos mediante el índice jerárquico HNSW (*Hierarchical Navigable Small World*).

2. **Búsqueda dispersa BM25:** Complementa la anterior con una búsqueda léxica clásica que favorece coincidencias exactas de términos, especialmente útil para códigos de documento, siglas o identificadores.

5.3. Extracción Web (Scraping)

5.3.1. Motor de Renderizado con Playwright

La extracción web se implementa a través de un módulo basado en *Playwright*, un framework de automatización de navegadores desarrollado por Microsoft. A diferencia de soluciones simplistas basadas en peticiones HTTP, Playwright instancia un navegador Chromium completo en modo *headless* (sin interfaz gráfica), lo cual permite:

- Renderizar páginas JavaScript complejas (React, Vue, Angular).

- Ejecutar la carga diferida (*lazy loading*) de contenido.
- Resolver redirecciones y consentimientos de cookies.
- Capturar el DOM final tras la ejecución de todos los scripts.

El scraper incorpora rotación de *User-Agent*, reintentos con retroceso exponencial (*exponential backoff*) y una espera inteligente configurable para aplicaciones de una sola página (SPA). Tras obtener el HTML renderizado, se aplica una cadena de extracción de contenido compuesta por tres estrategias de *fallback*:

1. **Trafilatura:** Algoritmo NLP especializado en extraer contenido principal de páginas web eliminando menús, cabeceras, pies de página y publicidad. Se prioriza por su alta precisión.
2. **Readability:** Algoritmo alternativo basado en heurísticas de densidad de texto, comparable al modo lectura de los navegadores modernos.
3. **BeautifulSoup:** Extracción cruda de todo el texto visible del DOM como última alternativa si las dos anteriores no producen un resultado superior al umbral mínimo de 200 caracteres.

5.3.2. Búsqueda en Internet

Complementariamente al scraping de URLs específicas, el sistema ofrece un modo de búsqueda general en internet. Mediante la librería `duckduckgo_search` y un *fallback* propio de scraping HTML directo sobre el dominio `html.duckduckgo.com`, se ejecutan consultas en lenguaje natural y se devuelven los resultados más relevantes con sus enlaces directos, evitando redirects intermedios.

5.4. Reconocimiento Óptico de Caracteres (OCR)

5.4.1. Motor PaddleOCR con Aceleración GPU

El motor de OCR seleccionado es **PaddleOCR**, desarrollado por Baidu, que ofrece reconocimiento de texto en más de 80 idiomas con soporte de ejecución acelerada por GPU mediante CUDA. Se ejecuta sobre las tarjetas NVIDIA del servidor utilizando los núcleos CUDA para el procesamiento paralelo de las imágenes.

La inicialización del motor implementa un mecanismo de *fallback* automático: si la GPU no está disponible o falla la carga de los controladores CUDA, se reinicia automáticamente en modo CPU sin intervención manual:

```python
try:
    self.ocr = PaddleOCR(
        use_angle_cls=True,
        lang="latin",
        use_gpu=True,
        gpu_mem=500
    )
except Exception as e:
    logger.warning(f"Fallo GPU, reintentando en CPU: {e}")
    self.ocr = PaddleOCR(
        use_angle_cls=True,
        lang="latin",
        use_gpu=False
    )
```

Listing 5.3: Inicialización de PaddleOCR con fallback CPU/GPU.

5.4.2. Flujo de Procesamiento de PDFs Escaneados

Cuando el backend recibe un archivo PDF, ejecuta un proceso de detección inteligente en dos fases:

1. **Extracción textual directa:** Se intenta obtener texto mediante bibliotecas de parseo de PDF como `pdfplumber`. Si el volumen de texto extraído es satisfactorio (> 100 caracteres por página), se asume que el documento es nativo digital y se procesa directamente.

2. **Fallback OCR:** Si la extracción directa produce un resultado insuficiente (indicador de que se trata de un PDF escaneado), cada página se convierte a imagen y se pasa por el motor PaddleOCR, que detecta los cuadros delimitadores de texto, los clasifica angularmente y transcribe su contenido.

5.5. Integración con Microsoft SharePoint

El módulo **Indexer** funciona como un servicio autónomo (*worker*) que ejecuta un ciclo de sincronización periódica cada 5 minutos mediante un cron interno. Su objetivo es mantener actualizada la base de conocimiento del RAG con los documentos corporativos alojados en SharePoint.

5.5.1. Autenticación MSAL y Microsoft Graph

La conexión con SharePoint se realiza mediante el protocolo OAuth 2.0 con flujo de credenciales de cliente (*Client Credentials Flow*), implementado a través de la librería MSAL (*Microsoft Authentication Library*). El servicio obtiene un token de acceso del *tenant* de Azure AD y utiliza la API REST de Microsoft Graph para listar y descargar los ficheros de los sitios configurados.

5.5.2. Sincronización Delta Incremental

Para evitar reprocesar documentos ya ingeridos, el Indexer implementa un mecanismo de sincronización delta: consulta la marca temporal de modificación de cada fichero en SharePoint y la compara con los registros almacenados en PostgreSQL. Solo los ficheros nuevos o modificados desde la última ejecución son descargados, procesados (chunking + embeddings) e indexados en Qdrant.

5.6. Servidor del Boletín Oficial del Estado (BOE)

5.6.1. Implementación del Protocolo MCP

La integración con el BOE se implementa utilizando el *Model Context Protocol* (MCP), un estándar abierto propuesto por Anthropic para formalizar la comunicación entre modelos de lenguaje y herramientas externas. Se despliega un servidor MCP local construido con la librería `fastmcp`.

El servidor expone funciones de búsqueda y consulta que transforman las peticiones en lenguaje natural del usuario en llamadas estructuradas a la API de Datos Abiertos del BOE Español.

5.6.2. Resolución Semántica de Leyes

Para consultas del tipo “Resume la Ley de Protección de Datos”, el sistema implementa un mecanismo de resolución semántica que mapea nombres coloquiales a identificadores oficiales del BOE:

```
LAW_MAPPINGS = {
    "proteccion de datos": "BOE-A-2018-16673",  # LOPDGDD
    "constitucion": "BOE-A-1978-31229",
    "propiedad intelectual": "BOE-A-1996-8930",
    "igualdad": "BOE-A-2007-6115",
```

```
}
```

Listing 5.4: Mapeo de nombres comunes a identificadores BOE.

El resolvedor elimina palabras vacías (*stop words*) comunes del español ("la", "de", "del", "ley") y busca coincidencias parciales en el diccionario de mapeos antes de acudir al *endpoint* de búsqueda general del BOE.

5.7. Fine-Tuning con LoRA

5.7.1. Generación del Dataset de Entrenamiento

Para adaptar el modelo base a las necesidades específicas del dominio corporativo, se implementó un pipeline de generación automática de datos de entrenamiento. Un script extrae pares pregunta-respuesta directamente de los fragmentos indexados en Qdrant, utilizando el modelo base LLaMA 3.1 como generador sintético de consultas plausibles para cada fragmento.

5.7.2. Entrenamiento con Low-Rank Adaptation

El ajuste fino se realiza mediante la técnica LoRA (*Low-Rank Adaptation*), que inserta matrices de rango reducido en las capas de atención del transformador sin modificar los pesos originales del modelo. Esto permite un entrenamiento eficiente con requisitos de VRAM significativamente inferiores al ajuste fino completo.

El modelo resultante, `rag-qwen-ft:latest`, se basa en Qwen 2.5 y se registra en LiteLLM como el modelo primario bajo el alias "JARVIS" para todas las consultas RAG corporativas.

Tabla 5.3: Modelos de IA desplegados en el sistema y su configuración.

Modelo	**Cuantización**	**VRAM**	**Uso**
LLaMA 3.1 8B Instruct	Q8_0	∼8 GB	Chat general y RAG
LLaMA 3.1 8B Instruct	Q4_0	∼4 GB	Resúmenes rápidos
Qwen 2.5 (LoRA FT)	—	∼5 GB	Respuestas corporativas
Qwen 2.5 VL 7B	—	∼5 GB	Análisis de imágenes
LLaVA 13B	—	∼8 GB	OCR visual (legacy)
MiniLM-L12-v2	FP32	CPU only	Embeddings 384d
PaddleOCR	—	∼500 MB	Reconocimiento de texto

5.8. Resultados y Evaluación

5.8.1. Métricas de Rendimiento

El sistema fue sometido a pruebas de rendimiento sostenidas en un entorno de producción real. La Tabla **??** sintetiza las métricas principales recogidas por Prometheus durante un período de operación continuada.

Tabla 5.4: Métricas de rendimiento del sistema en producción.

Métrica	**Valor**
Tiempo hasta primer token (TTFT)	$< 1{,}2$ s
Latencia RAG completa (p95)	$\sim 3{,}5$ s
Throughput máximo sostenido	~ 42 req/min
Uptime del sistema	99.8 % (30 días)
Consumo VRAM pico (3 modelos)	18.5 / 24 GB
Alucinaciones en modo RAG	0 % (temp ≤ 0.2)
Cache hit rate (Redis)	~ 35 %

5.8.2. Evaluación Cualitativa

Se realizaron pruebas funcionales abarcando los siguientes escenarios operativos:

1. **Consulta RAG sobre documentos corporativos:** El usuario pregunta "¿Cuál es la política de calidad de la empresa?" y el sistema recupera los fragmentos más relevantes del PDF de calidad indexado, generando una respuesta fundamentada exclusivamente en el contenido real del documento.

2. **Scraping de URL:** El usuario envía una URL y el sistema la descarga, renderiza con Playwright, limpia con Trafilatura, la trocea en chunks y genera una respuesta en contexto inmediato, almacenando además el contenido para posteriores consultas.

3. **Búsqueda legal en el BOE:** El usuario solicita "Busca en el BOE la ley de protección de datos" y el agente resuelve semánticamente la ley, consulta la API oficial y devuelve un resumen estructurado.

4. **Análisis de imagen:** El usuario adjunta una fotografía de un documento en papel y el sistema la procesa con los modelos de visión (Qwen 2.5 VL o LLaVA) para describir o transcribir su contenido.

5.8.3. Mitigación de Alucinaciones

La eliminación de alucinaciones en modo RAG se logra mediante la combinación de dos técnicas:

- **Temperatura determinista:** Los parámetros de inferencia se configuran con valores de temperatura extremadamente bajos (`0.1`–`0.2`), minimizando la aleatoriedad en la selección de tokens.
- **Prompt de sistema restrictivo:** El *system prompt* inyectado instruye explícitamente al modelo para que responda **únicamente** basándose en el contexto recuperado, prohibiendo la generación libre de información no respaldada por los fragmentos.

El solapamiento de ventanas contextuales (*overlap*) del 10–15 % en el chunking complementa estas medidas, asegurando que el hilo semántico de párrafos extensos no se pierda en los límites de fragmento.

Capítulo 6

Conclusiones y Trabajo Futuro

6.1. Resumen General

El presente Trabajo de Fin de Grado ha tenido como objetivo el diseño, desarrollo y despliegue de un sistema de Generación Aumentada por Recuperación (RAG) de ámbito empresarial, desplegado íntegramente en infraestructura local y construido exclusivamente con tecnologías de código abierto. El resultado ha sido un ecosistema funcional, compuesto por 12 microservicios contenerizados, capaz de ingerir documentos corporativos desde múltiples fuentes, procesarlos semánticamente y ofrecer a los usuarios una interfaz conversacional inteligente para consultar su base de conocimiento.

El sistema demuestra que la orquestación distribuida de múltiples modelos de Inteligencia Artificial (LLMs, Embeddings y Motores OCR visuales) ejecutados localmente no solamente viabiliza entornos de trabajo corporativos sumamente avanzados, sino que a su vez elimina por completo la dependencia de proveedores externos y los riesgos de fuga de datos asociados.

6.2. Consecución de Objetivos

A continuación, se evalúa el grado de cumplimiento de cada uno de los objetivos específicos planteados en el Capítulo **??**:

1. **Diseño de Arquitectura Desacoplada** — *Cumplido.* Se ha implementado una arquitectura de microservicios completa, empaquetada en 12 contenedores Docker orquestados mediante un único manifiesto `docker-compose.yml`. La separación de responsabilidades permite sustituir cualquier componente (modelo de IA, base de datos, proxy) sin afectar al resto del sistema.

2. **Integración Multicanal** — *Cumplido.* El sistema ingiere documentos desde tres fuentes heterogéneas: (a) Microsoft SharePoint mediante la API Graph con autenticación MSAL y sincronización delta incremental; (b) Web scraping con Playwright y limpieza con Trafilatura; (c) Carga manual de ficheros con OCR automático vía PaddleOCR cuando se detectan PDFs escaneados.

3. **Resolución Normativa** — *Cumplido parcialmente.* Se ha implementado un servidor MCP funcional que conecta con la API del BOE, con resolución semántica de nombres de leyes comunes. No obstante, la integración se considera un prototipo técnico dado que la cobertura legislativa se limita a un diccionario estático de leyes conocidas.

4. **Enrutamiento Inteligente** — *Cumplido.* El agente JARVIS detecta 9 modos de operación distintos (RAG, web search, scraping, visión, BOE, status, help, listado de documentos y conversación general) mediante heurísticas semánticas evaluadas en orden de prioridad estricto.

5. **Despliegue y Monitorización** — *Cumplido.* El sistema se despliega en un servidor on-premise con stack completo de observabilidad: Prometheus para métricas, NVIDIA DCGM Exporter para telemetría GPU, Grafana para dashboards visuales y pgAdmin para auditoría de la base de datos relacional.

6.2.1. Funcionalidades Logradas

La Tabla **??** resume la totalidad de funcionalidades implementadas en el sistema:

Tabla 6.1: Checklist de funcionalidades logradas.

Funcionalidad	**Estado**
Conversación general con LLMs locales	✓
RAG documental con búsqueda híbrida	✓
Web scraping con renderizado JavaScript	✓
Búsqueda en internet (DuckDuckGo)	✓
OCR de PDFs escaneados (GPU)	✓
Análisis de imágenes (visión multimodal)	✓
Consulta legislativa BOE (MCP)	✓
Sincronización automática SharePoint	✓
Control de acceso departamental (Azure AD)	✓
Monitorización GPU y métricas	✓
Caché de respuestas (Redis)	✓
Fine-tuning LoRA del modelo corporativo	✓
Autoayuda ("¿Qué puedes hacer?")	✓
Estado de indexación ("¿Cómo va?")	✓

6.3. Comparación con Otros Sistemas

Para contextualizar las contribuciones de este proyecto, se presenta una comparación con soluciones RAG comerciales y de código abierto relevantes:

Tabla 6.2: Comparación del sistema con soluciones RAG alternativas.

Característica	**Nuestro**	**ChatGPT**	**Copilot**	**LangChain**	**PrivateGPT**
On-Premise 100 %	✓	×	×	✓	✓
Código abierto	✓	×	×	✓	✓
Multi-fuente RAG	✓	Parcial	Parcial	✓	Parcial
OCR integrado	✓	×	×	×	×
Web Scraping	✓	×	Parcial	✓	×
Monitorización GPU	✓	×	×	×	×
SSO corporativo	✓	×	✓	×	×
Servidor MCP (BOE)	✓	×	×	×	×
UI conversacional	✓	✓	✓	×	✓
Coste por token	0 €	$$$	$$$	Variable	0 €

La principal ventaja diferencial del sistema desarrollado frente a soluciones co-

merciales como ChatGPT o Microsoft Copilot es la **soberanía total sobre los datos**: ningún documento, consulta o respuesta abandona el perímetro del servidor corporativo. Frente a frameworks de código abierto como LangChain, la ventaja reside en ofrecer un **sistema llave en mano** completamente desplegable, con interfaz de usuario, autenticación corporativa y monitorización incluidas de serie.

6.4. Lecciones Aprendidas

El desarrollo del proyecto ha conllevado numerosos retos técnicos cuya resolución ha aportado un conocimiento práctico de alto valor. Se destacan las lecciones más significativas:

- **Gestión de VRAM como recurso escaso:** La GPU se convierte en el cuello de botella principal cuando se ejecutan simultáneamente múltiples modelos de lenguaje, modelos de visión y el motor OCR. La cuantización (Q8/Q4) y la estrategia de carga bajo demanda de Ollama resultaron indispensables para mantener el sistema operativo dentro de los 24 GB de VRAM disponibles.

- **La calidad del chunking determina la calidad del RAG:** Se comprobó experimentalmente que la segmentación de documentos en fragmentos demasiado pequeños (< 256 tokens) produce respuestas fragmentadas y descontextualizadas, mientras que fragmentos demasiado grandes (> 1536 tokens) diluyen la relevancia semántica. El rango óptimo se estableció entre 512 y 1024 tokens con un solapamiento del 10–15 %.

- **La temperatura del modelo es crítica en modo RAG:** Valores de temperatura superiores a 0.3 en el modo RAG producían respuestas creativas que añadían información no presente en el contexto recuperado. Fijar la temperatura a 0.1–0.2 eliminó completamente las alucinaciones en consultas documentales.

- **El scraping web requiere un enfoque por capas:** Ninguna estrategia de extracción funciona universalmente para todas las páginas web. La implementación de una cadena de fallback (Trafilatura $\rightarrow$ Readability $\rightarrow$ BeautifulSoup) con umbrales mínimos de calidad demostró ser la solución más robusta.

- **La autenticación SSO simplifica, pero no trivializa:** La integración con Azure AD vía OIDC requirió un esfuerzo significativo de configuración (registros de aplicación, permisos de Graph API, mapeo de grupos a colecciones),

pero una vez establecida, eliminó la fricción de gestión de usuarios y simplificó enormemente el control de acceso departamental.

6.5. Estimación de Dedicación Total

Tabla 6.3: Distribución de horas dedicadas al proyecto.

Concepto	**Horas**
Desarrollo técnico (5 sprints)	160 h
Aprendizaje y pruebas previas	30 h
Redacción y maquetación de la memoria	40 h
Total estimado	**230 horas**

6.6. Trabajo Futuro y Líneas de Expansión

A pesar de la exhaustividad del proyecto, el ámbito de la Inteligencia Artificial evoluciona dinámicamente y expone ciertas vías de mejora continua:

1. **Ajuste Fino Avanzado (*Fine-Tuning*):** Aplicar estrategias estáticas de calibración paramétrica eficiente, tales como Low-Rank Adaptation (LoRA), adaptando dominios específicos (jerga interna empresarial) a los modelos instructivos LLaMA para depurar aún más el vocabulario de las respuestas. Adicionalmente, explorar técnicas de DPO (*Direct Preference Optimization*) utilizando el historial de conversaciones reales como datos de preferencia.

2. **Indexación de Contenido Multimedia:** Ampliar los parsers del backend actual al espectro de hojas de cálculo (*CSV/Excel*), PDFs e imágenes, sino acometer tareas de vectorización a nivel del audio corporativo con modelos *Whisper* o motores combinados de fonética multimodal.

3. **Evaluación Cuantitativa Formal:** Implementar benchmarks académicos estándar para la evaluación de sistemas RAG, como RAGAS (*Retrieval Augmented Generation Assessment*) **es2023ragas**, que proporcionarían métricas formales de fidelidad, relevancia y cobertura del contexto recuperado.

4. **Federación Multi-Agent:** Diseñar la arquitectura orquestal de JARVIS para segmentar de forma atómica sus *prompts*. Emplear redes recursivas de varios modelos reducidos cooperando para votar a consenso cuál es la mejor resolución al tratar interacciones altamente complejas.

5. **Integración Completa con Múltiples IDPs:** Avanzar sobre la presente Autenticación Única centrada en OIDC Azure AD para brindar cobertura horizontal a otros *Identity Providers* (Google Workspace, Okta, Keycloak), incrementando el Control de Acceso Basado en Roles (RBAC) para denegar el acceso semántico en las bases sectorizadas.

6. **Interfaz Móvil y API Pública:** Desarrollar una aplicación móvil nativa o una Progressive Web App (PWA) que consuma la API REST del backend, democratizando el acceso al asistente fuera de la estación de trabajo de la oficina.

6.7. Impacto y Transferencia

El sistema desarrollado en este TFG demuestra que es viable construir un asistente conversacional corporativo de alto rendimiento utilizando exclusivamente software de código abierto y hardware *commodity*. Este hallazgo tiene implicaciones directas para:

- **Pequeñas y medianas empresas (PYMEs):** Que pueden desplegar su propia infraestructura de IA sin incurrir en costes recurrentes de servicios cloud ni comprometer la confidencialidad de sus datos operacionales.

- **Administración Pública:** Donde las restricciones del RGPD y la Ley de Protección de Datos impiden el envío de datos ciudadanos a servidores fuera del Espacio Económico Europeo.

- **Sector Defensa e Industria:** Donde la soberanía del dato no es negociable y la infraestructura debe operar en redes aisladas sin conexión a internet.

La publicación íntegra del código fuente en GitHub[1] bajo licencia abierta facilita la replicabilidad del proyecto y su adaptación a otros dominios y organizaciones, contribuyendo al ecosistema de conocimiento abierto en el campo de la IA empresarial aplicada.

[1] https://github.com/acidxlemons/TFG-JARVIS